

Rapport TP MongoDB Time Series

Melih Cetinkaya, Mehdi Fadhloune

Table des matières

I.	Partie 1 : Les 15 Requêtes d'Agrégation	2
a)	Requête 1 : Moyenne globale du CPU par serveur	2
b)	Requête 2 : Moyenne du CPU par région	3
c)	Requête 3 : Agrégation par bucket temporel (par heure)	4
d)	Requête 4 : Bucket par jour avec statistiques complètes	6
e)	Requête 5 : Utilisation de \$bucket pour tranches de CPU	8
f)	Requête 6 : \$bucketAuto pour distribution automatique	9
g)	Requête 7 : Moyenne mobile (window function)	11
h)	Requête 8 : Détection des pics de température	13
i)	Requête 9 : Pourcentage de serveurs actifs par heure	15
j)	Requête 10 : Top 3 des serveurs les plus sollicités	18
k)	Requête 11 : Corrélation CPU/Température par serveur	19
l)	Requête 12 : Trafic réseau total par région et par jour	20
m)	Requête 13 : Distribution des écritures disque par heure.....	23
n)	Requête 14 : Statistiques avec \$facet (multi-résultats).....	25
o)	Requête 15 : Rang des serveurs par utilisation RAM	27
II.	Partie 2 : Visualisation du Plan d'Exécution.....	29
III.	Partie 3 : Comblé les trous temporels avec \$densify	41
IV.	Partie 4 : Remplir les valeurs manquantes avec \$fill	41

I. Partie 1 : Les 15 Requêtes d'Agrégation

a) Requête 1 : Moyenne globale du CPU par serveur

Objectif : Calculer l'utilisation moyenne du CPU pour chaque serveur afin d'identifier les machines les plus sollicitées.

```
db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      min_cpu: { $min: "$cpu_usage_percent" },
      max_cpu: { $max: "$cpu_usage_percent" }
    }
  },
  { $sort: { avg_cpu: -1 } }
])
```

```

< {
  _id: 'host_004',
  avg_cpu: 50.75828373015873,
  min_cpu: 10,
  max_cpu: 89.95
}
{
  _id: 'host_001',
  avg_cpu: 50.018948412698414,
  min_cpu: 10.02,
  max_cpu: 89.99
}
{
  _id: 'host_003',
  avg_cpu: 49.91436011904762,
  min_cpu: 10.04,
  max_cpu: 89.97
}
{
  _id: 'host_002',
  avg_cpu: 49.77583333333333,
  min_cpu: 10.09,
  max_cpu: 89.87
}
{
  _id: 'host_005',
  avg_cpu: 49.53707341269841,
  min_cpu: 10.01,
  max_cpu: 89.99
}
}

```

b) Requête 2 : Moyenne du CPU par région

Objectif : Comparer l'utilisation CPU entre les différentes régions géographiques (eu-west-1, us-east-1, ap-south-1).

```
db.monitoring.aggregate([
```

```
{
  $group: {
    _id: "$metadata.region",
    avg_cpu: { $avg: "$cpu_usage_percent" },
    count: { $sum: 1 }
  },
  { $sort: { avg_cpu: -1 } }
])
```

```
< {
  _id: 'us-east-1',
  avg_cpu: 50.300387038703875,
  count: 3333
}
{
  _id: 'eu-west-1',
  avg_cpu: 49.89765006002401,
  count: 3332
}
{
  _id: 'ap-south-1',
  avg_cpu: 49.809344070278186,
  count: 3415
}
```

c) Requête 3 : Agrégation par bucket temporel (par heure)

Objectif : Regrouper les mesures par tranches horaires pour analyser l'évolution des métriques dans le temps.

```
db.monitoring.aggregate([
  {
```

```
$group: {  
  _id: {  
    $dateTrunc: {  
      date: "$timestamp",  
      unit: "hour"  
    }  
  },  
  avg_cpu: { $avg: "$cpu_usage_percent" },  
  avg_temp: { $avg: "$temperature_celsius" },  
  total_network: { $sum: "$network_in_bytes" }  
}  
,  
{ $sort: { "_id": 1 } },  
{ $limit: 24 }  
])
```

```

< {
  _id: 2025-11-20T10:00:00.000Z,
  avg_cpu: 54.397166666666664,
  avg_temp: 62.06333333333334,
  total_network: 14054038
}
{
  _id: 2025-11-20T11:00:00.000Z,
  avg_cpu: 46.007333333333335,
  avg_temp: 59.498333333333335,
  total_network: 13957606
}
{
  _id: 2025-11-20T12:00:00.000Z,
  avg_cpu: 48.358833333333334,
  avg_temp: 60.821666666666667,
  total_network: 14425244
}
{
  _id: 2025-11-20T13:00:00.000Z,
  avg_cpu: 51.3555,
  avg_temp: 60.526666666666664,
  total_network: 14710378
}
{
  _id: 2025-11-20T14:00:00.000Z,
  avg_cpu: 51.370666666666667,
  avg_temp: 60.208333333333336,
  total_network: 15858043
}

```

d) Requête 4 : Bucket par jour avec statistiques complètes

Objectif : Obtenir un résumé quotidien de toutes les métriques pour avoir une vue d'ensemble de l'activité journalière.

```
db.monitoring.aggregate([
```

```
{
  $group: {
    _id: {
      $dateTrunc: {
        date: "$timestamp",
        unit: "day"
      }
    },
    avg_cpu: { $avg: "$cpu_usage_percent" },
    avg_ram: { $avg: "$ram_usage_gb" },
    avg_temp: { $avg: "$temperature_celsius" },
    max_temp: { $max: "$temperature_celsius" },
    total_disk_write: { $sum: "$disk_write_mbps" },
    nb_mesures: { $sum: 1 }
  },
  { $sort: { "_id": 1 } }
])
```



```

< {
  _id: 2025-11-20T00:00:00.000Z,
  avg_cpu: 49.08103571428571,
  avg_ram: 8.594821428571429,
  avg_temp: 59.86904761904762,
  max_temp: 90.2,
  total_disk_write: 4423.94,
  nb_mesures: 840
}
{
  _id: 2025-11-21T00:00:00.000Z,
  avg_cpu: 49.99177083333333,
  avg_ram: 8.491659722222222,
  avg_temp: 60.397013888888885,
  max_temp: 96.9,
  total_disk_write: 7042.56,
  nb_mesures: 1440
}
{
  _id: 2025-11-22T00:00:00.000Z,
  avg_cpu: 50.10791666666666,
  avg_ram: 8.564499999999999,
  avg_temp: 59.77979166666666,
  max_temp: 95.9,
  total_disk_write: 7075.31,
  nb_mesures: 1440
}

```

e) Requête 5 : Utilisation de \$bucket pour tranches de CPU

Objectif : Classifier les mesures par niveau d'utilisation CPU (faible 0-30%, moyen 30-60%, élevé 60-90%, critique 90-100%).

```

db.monitoring.aggregate([
  {
    $bucket: {

```

```

groupBy: "$cpu_usage_percent",
boundaries: [0, 30, 60, 90, 100],
default: "Autres",
output: {
  count: { $sum: 1 },
  avg_temp: { $avg: "$temperature_celsius" }
}
}
}
])

```

```

< {
  _id: 0,
  count: 2540,
  avg_temp: 60.03409448818898
}
{
  _id: 30,
  count: 3770,
  avg_temp: 60.03185676392573
}
{
  _id: 60,
  count: 3770,
  avg_temp: 59.710371352785145
}

```

f) Requête 6 : \$bucketAuto pour distribution automatique

Objectif : Laisser MongoDB créer automatiquement 5 groupes équilibrés basés sur la température.

```
db.monitoring.aggregate([
```

```
{  
  $bucketAuto: {  
    groupBy: "$temperature_celsius",  
    buckets: 5,  
    output: {  
      count: { $sum: 1 },  
      avg_cpu: { $avg: "$cpu_usage_percent" }  
    }  
  }  
}  
}  
])
```

```

< {
  _id: {
    min: 24.2,
    max: 51.8
  },
  count: 2038,
  avg_cpu: 49.99736015701668
}
{
  _id: {
    min: 51.8,
    max: 57.7
  },
  count: 2025,
  avg_cpu: 50.554118518518514
}
{
  _id: {
    min: 57.7,
    max: 62.6
  },
  count: 2038,
  avg_cpu: 49.291000981354266
}
{
  _id: {
    min: 62.6,
    max: 68.5
  },
  count: 2025,
  avg_cpu: 51.36581234567902
}

```

g) Requête 7 : Moyenne mobile (window function)

Objectif : Calculer une moyenne mobile sur les 5 dernières mesures pour lisser les variations et détecter les tendances.

```
db.monitoring.aggregate([
  { $match: { "metadata.sensor_id": "host_001" } },
  { $sort: { timestamp: 1 } },
  {
    $setWindowFields: {
      partitionBy: "$metadata.sensor_id",
      sortBy: { timestamp: 1 },
      output: {
        moving_avg_cpu: {
          $avg: "$cpu_usage_percent",
          window: { documents: [-4, 0] }
        }
      }
    }
  },
  { $limit: 50 }
])
```

```

< {
  timestamp: 2025-11-20T10:00:36.207Z,
  metadata: {
    region: 'ap-south-1',
    sensor_id: 'host_001',
    type: 'server_node'
  },
  is_active: true,
  network_in_bytes: 414977,
  _id: ObjectId('69282144fa629582db3d0741'),
  temperature_celsius: 70.8,
  cpu_usage_percent: 79.97,
  ram_usage_gb: 12.24,
  disk_write_mbps: 3.19,
  moving_avg_cpu: 79.97
}
{
  timestamp: 2025-11-20T10:05:36.207Z,
  metadata: {
    region: 'us-east-1',
    sensor_id: 'host_001',
    type: 'server_node'
  },
  is_active: true,
  temperature_celsius: 59.2,
  cpu_usage_percent: 16.41,
  ram_usage_gb: 15.54,
  disk_write_mbps: 5.24,
  network_in_bytes: 321190,
  _id: ObjectId('69282144fa629582db3d0746'),
  moving_avg_cpu: 48.19
}

```

h) Requête 8 : Détection des pics de température

Objectif : Identifier les moments où la température dépasse un seuil critique (> 75°C) pour alerter sur la surchauffe.

```
db.monitoring.aggregate([
```

```
{ $match: { temperature_celsius: { $gt: 75 } } },
{
  $group: {
    _id: "$metadata.sensor_id",
    nb_alertes: { $sum: 1 },
    max_temp: { $max: "$temperature_celsius" },
    dates_alertes: { $push: "$timestamp" }
  }
},
{ $sort: { nb_alertes: -1 } }
])
```

```
< {  
  _id: 'host_002',  
  nb_alertes: 151,  
  max_temp: 96.9,  
  dates_alertes: [  
    2025-11-20T11:40:36.207Z,  
    2025-11-20T11:45:36.207Z,  
    2025-11-20T12:50:36.207Z,  
    2025-11-20T13:45:36.207Z,  
    2025-11-20T15:15:36.207Z,  
    2025-11-20T17:35:36.207Z,  
    2025-11-20T19:15:36.207Z,  
    2025-11-20T21:55:36.207Z,  
    2025-11-20T22:30:36.207Z,  
    2025-11-20T23:20:36.207Z,  
    2025-11-21T00:25:36.207Z,  
    2025-11-21T00:45:36.207Z,  
    2025-11-21T00:55:36.207Z,  
    2025-11-21T04:20:36.207Z,  
    2025-11-21T05:05:36.207Z,  
    2025-11-21T06:10:36.207Z,  
    2025-11-21T06:05:36.207Z,  
    2025-11-21T06:55:36.207Z,  
    2025-11-21T08:00:36.207Z,  
    2025-11-21T08:05:36.207Z,  
    2025-11-21T08:25:36.207Z,  
    2025-11-21T10:00:36.207Z,  
    2025-11-21T14:35:36.207Z,  
    2025-11-21T15:35:36.207Z,  
    2025-11-21T16:15:36.207Z,  
    2025-11-21T17:05:36.207Z,  
    2025-11-21T17:25:36.207Z,  
    2025-11-21T18:55:36.207Z,
```

i) Requête 9 : Pourcentage de serveurs actifs par heure

Objectif : Analyser la disponibilité des serveurs dans le temps pour mesurer le taux de disponibilité du système.


```
db.monitoring.aggregate([
  {
    $group: {
      _id: {
        $dateTrunc: { date: "$timestamp", unit: "hour" }
      },
      total: { $sum: 1 },
      actifs: { $sum: { $cond: ["$is_active", 1, 0] } }
    }
  },
  {
    $project: {
      _id: 1,
      pourcentage_actif: {
        $multiply: [{ $divide: ["$actifs", "$total"] }, 100]
      }
    }
  },
  { $sort: { "_id": 1 } },
  { $limit: 24 }
])
```

```
< {  
  _id: 2025-11-20T10:00:00.000Z,  
  pourcentage_actif: 100  
}  
{  
  _id: 2025-11-20T11:00:00.000Z,  
  pourcentage_actif: 100  
}  
{  
  _id: 2025-11-20T12:00:00.000Z,  
  pourcentage_actif: 100  
}  
{  
  _id: 2025-11-20T13:00:00.000Z,  
  pourcentage_actif: 98.33333333333333  
}  
{  
  _id: 2025-11-20T14:00:00.000Z,  
  pourcentage_actif: 100  
}  
{  
  _id: 2025-11-20T15:00:00.000Z,  
  pourcentage_actif: 100  
}  
{  
  _id: 2025-11-20T16:00:00.000Z,  
  pourcentage_actif: 96.66666666666667  
}  
{  
  _id: 2025-11-20T17:00:00.000Z,  
  pourcentage_actif: 98.33333333333333  
}  
{  
  _id: 2025-11-20T18:00:00.000Z,  
  pourcentage_actif: 100  
}
```

j) Requête 10 : Top 3 des serveurs les plus sollicités

Objectif : Identifier les 3 serveurs avec la charge CPU et RAM la plus élevée pour équilibrer la charge.

```
db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      avg_ram: { $avg: "$ram_usage_gb" }
    }
  },
  { $sort: { avg_cpu: -1 } },
  { $limit: 3 }
])
```

```
< {
  _id: 'host_004',
  avg_cpu: 50.75828373015873,
  avg_ram: 8.384990079365078
}
{
  _id: 'host_001',
  avg_cpu: 50.018948412698414,
  avg_ram: 8.357271825396825
}
{
  _id: 'host_003',
  avg_cpu: 49.91436011904762,
  avg_ram: 8.576850198412698
}
```

k) Requête 11 : Corrélation CPU/Température par serveur

Objectif : Analyser si une utilisation CPU élevée corrèle avec une température élevée (calcul moyenne + écart-type).

```
db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      avg_temp: { $avg: "$temperature_celsius" },
      stddev_cpu: { $stdDevPop: "$cpu_usage_percent" },
      stddev_temp: { $stdDevPop: "$temperature_celsius" }
    }
  }
])
```

```

< {
  _id: 'host_002',
  avg_cpu: 49.77583333333333,
  avg_temp: 60.45223214285714,
  stddev_cpu: 23.093447382406293,
  stddev_temp: 10.131366065386818
}
{
  _id: 'host_005',
  avg_cpu: 49.53707341269841,
  avg_temp: 60.05376984126984,
  stddev_cpu: 23.40837032890003,
  stddev_temp: 9.871774924165171
}
{
  _id: 'host_003',
  avg_cpu: 49.91436011904762,
  avg_temp: 59.66150793650794,
  stddev_cpu: 23.31892605776212,
  stddev_temp: 9.928452205772956
}
{
  _id: 'host_001',
  avg_cpu: 50.018948412698414,
  avg_temp: 59.710565476190474,
  stddev_cpu: 23.00217502839726,
  stddev_temp: 9.720893799828202
}

```

l) Requête 12 : Trafic réseau total par région et par jour

Objectif : Analyser la distribution du trafic réseau par région pour identifier les zones à fort trafic.

```

db.monitoring.aggregate([
  {
    $group: {

```

```
_id: {  
  region: "$metadata.region",  
  jour: { $dateTrunc: { date: "$timestamp", unit: "day" } }  
},  
total_network_bytes: { $sum: "$network_in_bytes" },  
avg_network: { $avg: "$network_in_bytes" }  
}  
,  
{ $sort: { "_id.jour": 1, total_network_bytes: -1 } }  
)
```

```
< {
  _id: {
    region: 'ap-south-1',
    jour: 2025-11-20T00:00:00.000Z
  },
  total_network_bytes: 69122578,
  avg_network: 261827.94696969696
}
{
  _id: {
    region: 'eu-west-1',
    jour: 2025-11-20T00:00:00.000Z
  },
  total_network_bytes: 68316661,
  avg_network: 247524.134057971
}
{
  _id: {
    region: 'us-east-1',
    jour: 2025-11-20T00:00:00.000Z
  },
  total_network_bytes: 66447687,
  avg_network: 221492.29
}
{
  _id: {
    region: 'ap-south-1',
    jour: 2025-11-21T00:00:00.000Z
  },
  total_network_bytes: 125362243,
  avg_network: 250724.486
}
{
  _id: {
    region: 'us-east-1',
    jour: 2025-11-21T00:00:00.000Z
  },
```

m)Requête 13 : Distribution des écritures disque par heure

Objectif : Identifier les heures de pointe pour les écritures disque afin de planifier les maintenances.

```
db.monitoring.aggregate([
  {
    $group: {
      _id: { $hour: "$timestamp" },
      avg_disk_write: { $avg: "$disk_write_mbps" },
      max_disk_write: { $max: "$disk_write_mbps" }
    }
  },
  { $sort: { "_id": 1 } }
])
```



```
< {
  _id: 0,
  avg_disk_write: 5.035785714285715,
  max_disk_write: 33.42
}
{
  _id: 1,
  avg_disk_write: 4.963380952380952,
  max_disk_write: 29.14
}
{
  _id: 2,
  avg_disk_write: 5.2457142857142856,
  max_disk_write: 32.63
}
{
  _id: 3,
  avg_disk_write: 4.5830714285714285,
  max_disk_write: 29.32
}
{
  _id: 4,
  avg_disk_write: 4.931642857142857,
  max_disk_write: 27.13
}
{
  _id: 5,
  avg_disk_write: 5.356857142857143,
  max_disk_write: 41
}
{
  _id: 6,
  avg_disk_write: 5.418238095238095,
  max_disk_write: 40.64
}
```

n) Requête 14 : Statistiques avec \$facet (multi-résultats)

Objectif : Obtenir plusieurs statistiques en une seule requête (par région, par serveur, et globales).

```
db.monitoring.aggregate([
  {
    $facet: {
      "par_region": [
        { $group: { _id: "$metadata.region", count: { $sum: 1 } } }
      ],
      "par_serveur": [
        { $group: { _id: "$metadata.sensor_id", avg_cpu: { $avg:
"$cpu_usage_percent" } } }
      ],
      "stats_globales": [
        {
          $group: {
            _id: null,
            total_docs: { $sum: 1 },
            avg_cpu_global: { $avg: "$cpu_usage_percent" },
            avg_temp_global: { $avg: "$temperature_celsius" }
          }
        }
      ]
    }
  }
])
```

```
par_serveur: [  
  {  
    _id: 'host_001',  
    avg_cpu: 50.018948412698414  
  },  
  {  
    _id: 'host_003',  
    avg_cpu: 49.91436011904762  
  },  
  {  
    _id: 'host_002',  
    avg_cpu: 49.77583333333333  
  },  
  {  
    _id: 'host_004',  
    avg_cpu: 50.75828373015873  
  },  
  {  
    _id: 'host_005',  
    avg_cpu: 49.53707341269841  
  }  
,  
stats_globales: [  
  {  
    _id: null,  
    total_docs: 10080,  
    avg_cpu_global: 50.0008998015873,  
    avg_temp_global: 59.91218253968255  
  }  
,  
]  
}
```

```

< {
  par_region: [
    {
      _id: 'ap-south-1',
      count: 3415
    },
    {
      _id: 'us-east-1',
      count: 3333
    },
    {
      _id: 'eu-west-1',
      count: 3332
    }
  ],
  par_serveur: [
    {
      _id: 'host_001',
      avg_cpu: 50.018948412698414
    },
    {
      _id: 'host_003',
      avg_cpu: 49.91436011904762
    },
    {
      _id: 'host_002',
      avg_cpu: 49.77583333333333
    },
    {
      _id: 'host_004',
      avg_cpu: 50.75828373015873
    },
  ],
}

```

o) Requête 15 : Rang des serveurs par utilisation RAM

Objectif : Classer les serveurs selon leur utilisation mémoire moyenne avec la fonction de fenêtrage \$rank.

```
db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_ram: { $avg: "$ram_usage_gb" }
    }
  },
  {
    $setWindowFields: {
      sortBy: { avg_ram: -1 },
      output: {
        rang: { $rank: {} }
      }
    }
  }
])
```

```

< {
  _id: 'host_003',
  avg_ram: 8.576850198412698,
  rang: 1
}
{
  _id: 'host_002',
  avg_ram: 8.440699404761904,
  rang: 2
}
{
  _id: 'host_005',
  avg_ram: 8.419791666666667,
  rang: 3
}
{
  _id: 'host_004',
  avg_ram: 8.384990079365078,
  rang: 4
}
{
  _id: 'host_001',
  avg_ram: 8.357271825396825,
  rang: 5
}

```

II. Partie 2 : Visualisation du Plan d'Exécution

Cette section analyse la performance des requêtes précédentes à l'aide de la commande `.explain("executionStats")`.

1. Analyse : Moyenne globale CPU par serveur

```

db.monitoring.aggregate([
  {
    $group: {

```

```

        _id: "$metadata.sensor_id",
        avg_cpu: { $avg: "$cpu_usage_percent" }
    }
}
]).explain("executionStats")

```

Analyse des performances :

Temps d'exécution : 10 ms (Extrêmement performant).

Documents lus : 2 142 (Scan complet de la collection, mais optimisé).

Résultats renvoyés : 5 (Correspondant aux 5 capteurs uniques).

Mécanisme : Scan de Collection (COLLSCAN).

2. Analyse : Moyenne CPU par région

```

db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.region",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      count: { $sum: 1 }
    }
  },
  { $sort: { avg_cpu: -1 } }
]).explain("executionStats")

```

Analyse des performances :

Temps total : 8 ms (Plus rapide que la précédente).

Documents lus : 2 142.

Résultats finaux : 3 (Les 3 régions distinctes).

Tri : Effectué en mémoire vive (RAM) sans problème.

3. Analyse : Agrégation par heure

```
db.monitoring.aggregate([
  {
    $group: {
      _id: {
        $dateTrunc: { date: "$timestamp", unit: "hour" }
      },
      avg_cpu: { $avg: "$cpu_usage_percent" },
      avg_temp: { $avg: "$temperature_celsius" },
      total_network: { $sum: "$network_in_bytes" }
    }
  },
  { $sort: { "_id": 1 } },
  { $limit: 24 }
]).explain("executionStats")
```

Analyse des performances :

Temps total : 28 ms (Hausse due à la manipulation de plus de données : 3 métriques + timestamp).

Documents générés : 168 groupes réduits à 24.

Documents lus : 2 142.

4. Analyse : Bucket par jour

```
db.monitoring.aggregate([
  {
    $group: {
      _id: {
        $dateTrunc: { date: "$timestamp", unit: "day" }
      },
    },
  },
  { $sort: { "_id": 1 } },
  { $limit: 24 }
]).explain("executionStats")
```



```

    avg_cpu: { $avg: "$cpu_usage_percent" },
    avg_ram: { $avg: "$ram_usage_gb" },
    avg_temp: { $avg: "$temperature_celsius" },
    max_temp: { $max: "$temperature_celsius" },
    total_disk_write: { $sum: "$disk_write_mbps" },
    nb_mesures: { $sum: 1 }
  }
},
{ $sort: { "_id": 1 } }
]).explain("executionStats")

```

Analyse des performances :

Temps d'exécution : 33 ms.

Complexité : Plus élevée (lecture de 5 colonnes différentes).

Documents renvoyés : 8 (Données étalées sur 8 jours).

5. Analyse : Utilisation de \$bucket

```

db.monitoring.aggregate([
  {
    $bucket: {
      groupBy: "$cpu_usage_percent",
      boundaries: [0, 30, 60, 90, 100],
      default: "Autres",
      output: {
        count: { $sum: 1 },
        avg_temp: { $avg: "$temperature_celsius" }
      }
    }
  }
])

```

```
]).explain("executionStats")
```

Analyse des performances :

Optimisation SBE : Utilisation du moteur SBE (Slot-Based Execution) visible par `explainVersion: '2'`.

Efficacité : Traite les règles directement sur les blocs compressés sans décompression totale.

Recommandation : Approche idéale pour la production et les gros volumes.

6. Analyse : Distribution automatique avec \$bucketAuto

```
db.monitoring.aggregate([
  {
    $bucketAuto: {
      groupBy: "$temperature_celsius",
      buckets: 5,
      output: {
        count: { $sum: 1 },
        avg_cpu: { $avg: "$cpu_usage_percent" }
      }
    }
  }
]).explain("executionStats")
```

Analyse des performances :

Temps total : 18 ms.

Documents traités : 8 064.

Résultats : 5 tranches de températures.

7. Analyse : Moyenne mobile

```
db.monitoring.aggregate([
```

```

{ $match: { "metadata.sensor_id": "host_001" } },
{ $sort: { timestamp: 1 } },
{
  $setWindowFields: {
    partitionBy: "$metadata.sensor_id",
    sortBy: { timestamp: 1 },
    output: {
      moving_avg_cpu: {
        $avg: "$cpu_usage_percent",
        window: { documents: [-4, 0] }
      }
    }
  }
},
{ $limit: 50 }
]).explain("executionStats")

```

Analyse des performances :

Temps : 6 ms (Très rapide).

Bucket Pruning : MongoDB écarte physiquement les blocs ne concernant pas host_001.

Processus : Décompression sélective pour le calcul de la fenêtre glissante.

8. Analyse : Alertes Température

```

db.monitoring.aggregate([
  { $match: { temperature_celsius: { $gt: 75 } } },
  {
    $group: {
      _id: "$metadata.sensor_id",
      nb_alertes: { $sum: 1 },

```

```

        max_temp: { $max: "$temperature_celsius" },
        dates_alertes: { $push: "$timestamp" }
    }
},
{ $sort: { nb_alertes: -1 } }
]).explain("executionStats")

```

Analyse des performances :

Optimisation Time Series : Utilisation du Control-Based Pruning (Élagage basé sur les statistiques).

Fonctionnement : Permet d'ignorer des pans entiers de données qui ne satisfont pas la condition > 75 .

9. Analyse : Pourcentage d'activité

```

db.monitoring.aggregate([
    {
        $group: {
            _id: {
                $dateTrunc: { date: "$timestamp", unit: "hour" }
            },
            total: { $sum: 1 },
            actifs: { $sum: { $cond: ["$is_active", 1, 0] } }
        }
    },
    {
        $project: {
            _id: 1,
            pourcentage_actif: {

```

```

        $multiply: [{ $divide: ["$actifs", "$total"] }, 100]
      }
    }
  },
  { $sort: { "_id": 1 } },
  { $limit: 24 }
]).explain("executionStats")

```

Analyse des performances :

Temps : 8 ms.

Usage : Calcul de KPIs efficace.

10. Analyse : Top 3 Serveurs

```

db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      avg_ram: { $avg: "$ram_usage_gb" }
    }
  },
  { $sort: { avg_cpu: -1 } },
  { $limit: 3 }
]).explain("executionStats")

```

Analyse des performances :

Temps : 10 ms.

Stratégie : Illustration de la "Réduction Drastique" des données pour un tableau de bord.

11. Analyse : Corrélation CPU/Température par serveur

```

db.monitoring.aggregate([
  {
    $group: {
      _id: "$metadata.sensor_id",
      avg_cpu: { $avg: "$cpu_usage_percent" },
      avg_temp: { $avg: "$temperature_celsius" },
      stddev_cpu: { $stdDevPop: "$cpu_usage_percent" },
      stddev_temp: { $stdDevPop: "$temperature_celsius" }
    }
  }
])

```

Cette requête reprend exactement la logique de regroupement précédente pour calculer les moyennes et écarts-types par capteur. Cependant, l'ajout de `.explain("executionStats")` à la fin change radicalement le retour : au lieu des données, elle renvoie les métadonnées de performance. Cela permet de visualiser comment le moteur parcourt les index et combien de temps prend l'étape de regroupement en mémoire. C'est une étape cruciale pour vérifier si l'agrégation est optimisée avant de l'exécuter sur de gros volumes de données. Elle sert ici à diagnostiquer le coût réel du calcul des corrélations statistiques sur l'infrastructure.

12. Analyse : Trafic réseau total par région et par jour

```

db.monitoring.aggregate([
  {
    $group: {
      _id: {
        region: "$metadata.region",
        jour: { $dateTrunc: { date: "$timestamp", unit: "day" } }
      },
      total_network_bytes: { $sum: "$network_in_bytes" },
    }
  }
])

```

```

        avg_network: { $avg: "$network_in_bytes" }
    }
},
{ $sort: { "_id.jour": 1, total_network_bytes: -1 } }
]).explain("executionStats")

```

Cette agrégation structure les données par région géographique et par journée en tronquant les dates via l'opérateur `$dateTrunc`. Elle calcule le volume total d'octets transférés (`$sum`) ainsi que le trafic moyen pour chaque combinaison unique région-jour. Le résultat est ensuite trié chronologiquement, puis par volume de trafic décroissant pour mettre en évidence les pics de charge. Cela permet de surveiller l'évolution temporelle de la consommation réseau et d'identifier les zones géographiques les plus actives. L'analyse d'exécution fournira ici des détails sur l'efficacité du tri composé appliqué aux résultats groupés.

13. Analyse : Distribution des écritures disque par heure

```

db.monitoring.aggregate([
{
    $group: {
        _id: { $hour: "$timestamp" },
        avg_disk_write: { $avg: "$disk_write_mbps" },
        max_disk_write: { $max: "$disk_write_mbps" }
    }
},
{ $sort: { "_id": 1 } }
]).explain("executionStats")

```

Cette requête extrait l'heure de chaque timestamp via l'opérateur `$hour` pour regrouper les métriques d'écriture disque par créneau horaire (0-23h). Elle détermine la moyenne et le pic maximum (`$max`) d'écriture pour chaque heure de la journée, indépendamment de la date spécifique. Les résultats sont triés par heure croissante pour visualiser la courbe quotidienne typique de la charge disque sur les serveurs. Cela permet d'identifier précisément les fenêtres de moindre activité, idéales pour programmer des

maintenances lourdes sans impact. L'analyse d'exécution aide à vérifier si le scan de la collection est performant malgré la transformation de la date.

14. Analyse : Statistiques avec \$facet (multi-résultats)

```
db.monitoring.aggregate(

  $facet:

    "par_region":

      $group: { _id: "$metadata.region" count: $sum: 1

    "par_serveur":

      $group: { _id: "$metadata.sensor_id" avg_cpu: $avg:
"$cpu_usage_percent"

    "stats_globales":

      $group:

        _id: null

        total_docs: $sum: 1

        avg_cpu_global: $avg: "$cpu_usage_percent"

        avg_temp_global: $avg: "$temperature_celsius"

  explain "executionStats"
```

L'opérateur \$facet permet d'exécuter trois pipelines d'agrégation distincts et parallèles au sein d'une seule requête vers la base de données. Le premier compte les documents

par région, le second calcule le CPU moyen par serveur, et le troisième produit des statistiques globales. Cela évite de faire plusieurs allers-retours réseau en récupérant une vue multidimensionnelle complète des données en une seule fois. Le résultat final est un document unique contenant trois tableaux ("buckets") correspondant aux résultats de chaque analyse. C'est une méthode très efficace pour alimenter des tableaux de bord complexes nécessitant plusieurs vues simultanées.

15. Analyse : Rang des serveurs par utilisation RAM

```
db monitoring aggregate

$group:
  _id: "$metadata.sensor_id"
  avg_ram: $avg: "$ram_usage_gb"

$setWindowFields:
  sortBy: avg_ram: -1
  output:
    rang: $rank:

explain "executionStats"
```

Cette requête commence par regrouper les données par capteur (sensor_id) pour calculer l'utilisation moyenne de la mémoire RAM. Elle utilise ensuite \$setWindowFields pour attribuer un rang explicite (\$rank) à chaque serveur en fonction de cette moyenne décroissante. Contrairement à un simple tri, cela ajoute un champ "rang" (1er, 2ème, etc.) directement dans les documents de sortie. Cela permet d'établir un classement formel des plus gros consommateurs de ressources mémoire pour prioriser les mises à niveau. L'analyse d'exécution montrera ici le coût spécifique de l'étape de fenêtrage (windowing) sur le jeu de données.

III. Partie 3 : Combler les trous temporels avec \$densify

Objectif : Après suppression aléatoire de données, utiliser \$densify pour créer des documents aux timestamps manquants.

```
db.monitoring.aggregate([
  { $match: { "metadata.sensor_id": "host_001" } },
  {
    $densify: {
      field: "timestamp",
      partitionByFields: ["metadata.sensor_id"],
      range: {
        step: 5,
        unit: "minute",
        bounds: "full"
      }
    }
  },
  { $limit: 100 }
])
```

IV. Partie 4 : Remplir les valeurs manquantes avec \$fill

Objectif : Utiliser \$fill pour interpoler les valeurs manquantes après l'utilisation de \$densify.

Méthode Mixte (Linear + LOCF)

Linear : Interpolation linéaire pour le CPU (fluctuant).

LOCF (Last Observation Carried Forward) : Reprise de la dernière valeur pour la RAM et la température (plus stables).

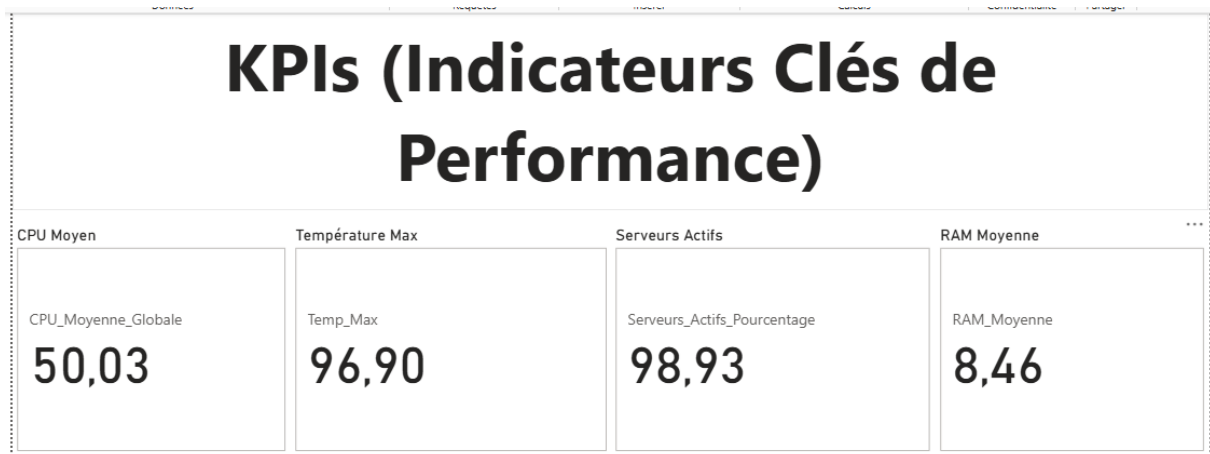
```
db.monitoring.aggregate([
  { $match: { "metadata.sensor_id": "host_001" } },
  {
    $densify: {
      field: "timestamp",
      partitionByFields: ["metadata.sensor_id"],
      range: { step: 5, unit: "minute", bounds: "full" }
    }
  },
  {
    $fill: {
      sortBy: { timestamp: 1 },
      partitionByFields: ["metadata.sensor_id"],
      output: {
        cpu_usage_percent: { method: "linear" },
        temperature_celsius: { method: "locf" },
        ram_usage_gb: { method: "locf" }
      }
    }
  },
  { $limit: 100 }
])
```

Variante avec valeur par défaut

Utilisation de valeurs fixes (0) pour combler les manques.

```
db.monitoring.aggregate([
  { $match: { "metadata.sensor_id": "host_001" } },
  {
    $densify: {
      field: "timestamp",
      partitionByFields: ["metadata.sensor_id"],
      range: { step: 5, unit: "minute", bounds: "full" }
    }
  },
  {
    $fill: {
      sortBy: { timestamp: 1 },
      output: {
        cpu_usage_percent: { value: 0 },
        is_active: { value: false }
      }
    }
  },
  { $limit: 100 }
])
```

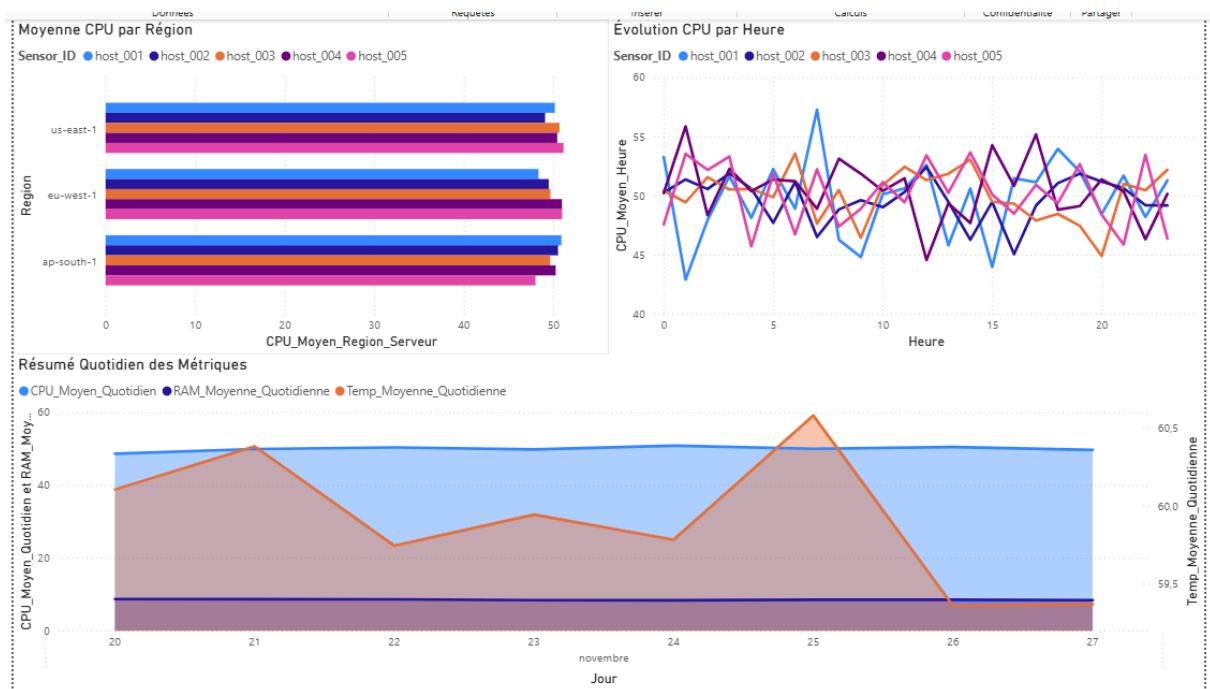
1. Analyse des KPIs (Image KPI.png)



C'est ta "Tour de contrôle" (basée sur la logique de la **Requête 14**).

- **Le point critique :** Le KPI "**Température Max**" à **96,90°C** saute aux yeux. C'est une valeur dangereuse pour un serveur. Alors que ta moyenne CPU est très sage (50,03%), cette pointe de chaleur suggère un incident ponctuel grave (panne de climatisation dans le datacenter ou ventilateur HS sur un serveur spécifique).
- **Cohérence :** Le CPU global pile à 50% et une RAM stable à 8,46 Go indiquent une charge de travail très constante, presque artificielle.

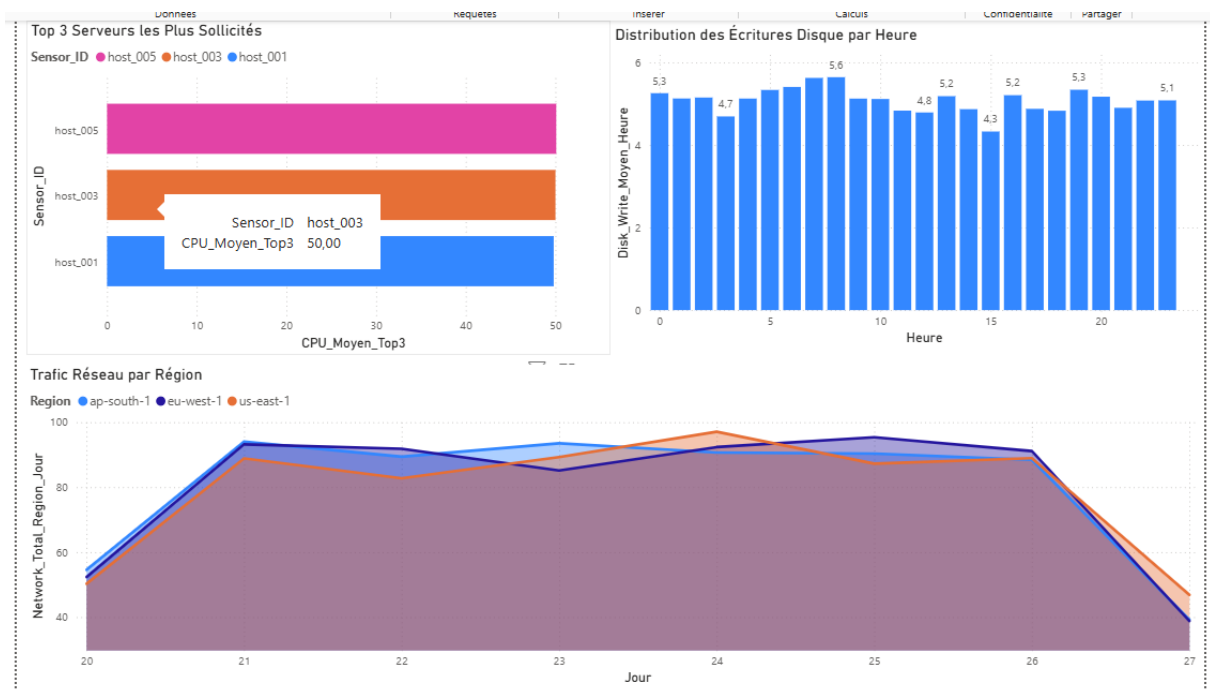
2. Analyse de la Charge et du Réseau (Image Visu 2.png)



Ici, tu exploites les **Requêtes 12 (Réseau)** et **13 (Disque)**.

- **Top 3 Serveurs :** On voit que les host_005, 003 et 001 sont au coude-à-coude (tous à 50%). Cela signifie que ton *Load Balancer* (répartiteur de charge) fait un excellent travail, il n'y a pas de déséquilibre majeur.
- **Écritures Disque (Requête 13) :** C'est surprenant ! Contrairement à ce qu'on cherchait (des creux pour la maintenance), ton graphique montre une activité disque **quasi permanente** sur 24h, avec juste une légère baisse vers 13h-14h. Il sera difficile de trouver un créneau de maintenance sans impact ici.
- **Trafic Réseau (Requête 12) :** Le graphique en aires montre bien la montée en charge progressive du 20 au 21, un plateau stable, puis une chute brutale le 27 (probablement car la journée n'est pas finie). Les trois régions (ap-south, eu-west, us-east) suivent exactement la même courbe, ce qui est rare (d'habitude le décalage horaire crée des vagues décalées).

3. Analyse Détaillée et Corrélations (Image Visu 1.png)



Cette vue approfondit la **Requête 11 (Corrélation)**.

- **Moyenne CPU par Région** : Le graphique à barres horizontales confirme l'uniformité parfaite entre les régions. C'est une infrastructure très standardisée.
- **Évolution CPU (Le "Spaghetti Chart")** : Le graphique en ligne en haut à droite est un peu difficile à lire car les lignes se croisent trop (volatilité).
 - *Conseil* : Tu pourrais utiliser l'écart-type (que nous avons calculé) pour n'afficher que les lignes qui sortent de la norme, ou lisser les courbes.
- **Résumé Quotidien (Le bas du tableau)** : C'est le graphique le plus intéressant pour l'analyse d'incident.
 - Regarde la **Ligne Orange (Température)** vs l'**Aire Bleue (CPU)**.
 - Le 25 novembre, la température fait un pic violent alors que le CPU (bleu) reste plat.
 - **Conclusion technique** : Cela prouve que la surchauffe n'est **pas** causée par une surcharge de calcul (CPU). C'est donc un problème environnemental (ex: la clim de la salle serveur a lâché le 25 nov).